

Preparation aux Entretiens

Algorithmes & Structures de donnees

Comment aborder un probleme de code,
les 21 patterns essentiels, et les pieges a eviter.

Table des matières

1	Methode : comment coder en entretien	2
2	Hash Map / Dictionnaire	3
3	Hash Set	3
4	Two Pointers (deux pointeurs)	4
5	Sliding Window (fenetre glissante)	4
6	Prefix Sum (somme prefixee)	5
7	Sorting (tri)	6
8	Binary Search (recherche dichotomique)	6
9	Stack (pile LIFO)	7
10	Monotonic Stack (pile monotone)	7
11	Backtracking (retour sur trace)	8
12	Linked List (liste chaine)	9
13	Fast & Slow Pointers (lievre et tortue)	9
14	Merge Intervals (fusion d'intervalles)	10
15	Cyclic Sort (tri cyclique)	11
16	BFS (parcours en largeur)	11
17	DFS (parcours en profondeur)	12
18	Topological Sort (tri topologique)	12
19	Union-Find / DSU (ensembles disjoints)	13
20	Dynamic Programming (programmation dynamique)	14
21	Greedy (algorithme glouton)	14
22	Bit Manipulation (manipulation de bits)	15
23	Fiche recapitulative des complexites	17

1. Methode : comment coder en entretien

Ne jamais foncer sur le clavier. Un entretien evalue autant ton *raisonnement* que ta solution. Suis systematiquement ces six etapes :

1. **Comprendre** – reformule l'enonce ; clarifie les contraintes (taille, valeurs, doublons, donnees trieées?).
2. **Exemples** – ecris 1 ou 2 exemples a la main, y compris les cas limites (tableau vide, un seul element).
3. **Force brute** – enonce la solution naive et sa complexite. Tu as toujours une base sur laquelle t'appuyer.
4. **Optimiser** – cherche le pattern : quelle structure de donnees ? quel invariant ? un tri prealable aide-t-il ?
5. **Coder** – ecris proprement en parlant a voix haute ; nomme clairement les variables.
6. **Tester** – deroule le code sur un exemple, verifie les cas limites, corrige.

Regle d'or

Annonce toujours la complexite (temps *et* espace) de ta solution, et propose une piste d'amelioration meme si tu n'as pas le temps de l'implementer.

2. Hash Map / Dictionnaire

Structure cle → valeur offrant un acces en $O(1)$ moyen. C'est l'outil le plus rentable en entretien : il transforme souvent un $O(n^2)$ en $O(n)$.

L'idée. Au lieu de re-parcourir tout le tableau a chaque fois qu'on cherche quelque chose, on *memorise au fur et a mesure* ce qu'on a deja vu dans un dictionnaire, puis on le consulte instantanément. On echange de la memoire ($O(n)$) contre du temps : c'est le compromis classique « espace contre vitesse ».

Reconnaitre le pattern. Des que l'enonce parle de *compter*, de *regrouper*, de *trouver une paire / un complement*, ou que ta premiere idee est une double boucle qui compare chaque element a tous les autres, pense hash map : elle supprime presque toujours la boucle interne.

Quand l'utiliser

Compter des frequences ; retrouver un complement deja vu (Two Sum) ; grouper des elements par cle ; memoriser des indices deja rencontres.

Complexite : Temps $O(1)$ moyen | Espace $O(n)$

Problemes types : Two Sum (LC 1), Group Anagrams (LC 49), Top K Frequent (LC 347).

```
def two_sum(nums, target):
    seen = {} # valeur -> indice
    for i, x in enumerate(nums):
        if target - x in seen:
            return [seen[target - x], i]
        seen[x] = i
    return []
```

Piege : en cas de collision de cles, la derniere valeur ecrase la precedente. Reflexis a ce que la cle doit reellement représenter.

3. Hash Set

Ensemble d'elements uniques ; test d'appartenance en $O(1)$ moyen.

L'idée. Un set est un hash map sans valeurs : on ne retient que la *presence* des elements. Il repond en temps constant a une seule question, mais la plus utile : « ai-je deja rencontre cet element ? ». On l'alimente au fil du parcours et on interroge avant d'ajouter.

Reconnaitre le pattern. Mots-cles declencheurs : *doublon*, *unique*, *deja vu*, *visite*. Si tu es tente d'ecrire `if x in ma_liste` a l'interieur d'une boucle (recherche en $O(n)$ a chaque tour, donc $O(n^2)$ au total), remplace la liste par un set et le cout tombe a $O(n)$.

Quand l'utiliser

Tester l'appartenance rapidement ; detecter les doublons ; suivre les elements deja visites (parcours de graphe, cycles).

Complexite : Temps $O(1)$ moyen | Espace $O(n)$

Problemes types : Contains Duplicate (LC 217), Longest Consecutive (LC 128), Happy Number (LC 202).

```
def has_duplicate(nums):
    seen = set()
    for x in nums:
        if x in seen:
            return True
    seen.add(x)
```

```
return False
```

Piege : un set ne conserve pas l'ordre d'insertion ; si l'ordre compte, utilise un dictionnaire (ordonne depuis Python 3.7) ou une liste annexe.

4. Two Pointers (deux pointeurs)

Deux indices qui avancent selon une regle. Deux variantes : *opposes* (left/right) ou *memme direction* (lent/rapide).

L'idée. Beaucoup de problemes qui semblent demander deux boucles imbriquees peuvent se resoudre avec deux curseurs qui balaient le tableau en un seul passage. Dans la variante *opposee*, on part des deux extremités d'un tableau *trie* et on rapproche les curseurs selon que la somme est trop grande ou trop petite. Dans la variante *lent/rapide*, un curseur avance plus vite que l'autre (utile pour les cycles ou le milieu d'une liste).

Reconnaitre le pattern. Tableau ou chaine *trie*, recherche d'une *paire* qui verifie une condition, *palindrome*, ou l'envie d'ecrire une double boucle $O(n^2)$ sur des donnees ordonnees : les deux pointeurs ramènent souvent le cout a $O(n)$ et $O(1)$ d'espace.

Quand l'utiliser

Tableau *trie* et recherche d'une paire ; palindrome ou inversion ; detection de cycle et recherche du milieu (lent/rapide).

Complexite : Temps $O(n)$ | Espace $O(1)$

Problemes types : Valid Palindrome (LC 125), Two Sum II (LC 167), 3Sum (LC 15).

```
def two_sum_sorted(nums, target):
    left, right = 0, len(nums) - 1
    while left < right:
        s = nums[left] + nums[right]
        if s == target:
            return [left, right]
        if s < target:
            left += 1
        else:
            right -= 1
    return []
```

Piege : la variante opposee suppose un tableau *trie*. Si l'enonce ne le garantit pas, il faut trier d'abord (+ $O(n \log n)$).

5. Sliding Window (fenetre glissante)

Maintenir une fenetre `[left, right]` et l'ajuster selon une contrainte, au lieu de recalculer chaque sous-tableau.

L'idée. Verifier tous les sous-tableaux coute $O(n^2)$. La fenetre glissante evite ce gaspillage : quand on avance le bord droit, on *ajoute* un element ; quand la contrainte est violee, on avance le bord gauche pour *retirer* des elements. Chaque element entre et sort de la fenetre au plus une fois, d'ou un cout total de $O(n)$. C'est comme une chenille qui s'etire et se contracte le long du tableau.

Reconnaitre le pattern. L'enonce demande le *plus long* / *plus court* sous-tableau ou sous-chaine *contigu* respectant une condition (somme, nombre de caracteres distincts, etc.), ou une fenetre de *taille fixe k*. Le mot cle est **contigu** : si les elements doivent etre voisins, pense fenetre glissante.

Quand l'utiliser

Sous-tableau ou sous-chaine *contigu* ; longueur min/max sous contrainte ; fenetre de taille fixe (somme de k elements consecutifs).

Complexite : Temps $O(n)$ | Espace $O(1)$ ou $O(k)$

Problemes types : Longest Substring (LC 3), Min Size Subarray (LC 209), Find Anagrams (LC 438).

```
def longest_unique(s):
    last = {}                                # caractere -> dernier indice vu
    left = best = 0
    for right, ch in enumerate(s):
        if ch in last and last[ch] >= left:
            left = last[ch] + 1
        last[ch] = right
        best = max(best, right - left + 1)
    return best
```

Piege : bien decider quand *agrandir* (avancer **right**) et quand *retrecir* (avancer **left**) la fenetre ; un ordre incorrect casse l'invariant.

6. Prefix Sum (somme prefixee)

On precalcule `prefix[i] = somme de nums[0..i-1]`. Toute somme d'intervalle se lit alors en $O(1)$: `sum(i, j) = prefix[j+1] - prefix[i]`.

L'idée. Repondre a beaucoup de questions « somme entre i et j » en re-additionnant a chaque fois coute cher. En calculant *une seule fois* les sommes cumulees depuis le debut, toute somme partielle devient une simple soustraction de deux valeurs deja connues. On investit $O(n)$ au depart pour que chaque requete suivante soit gratuite.

Reconnaitre le pattern. *Nombreuses* requetes de somme (ou de compte) sur des intervalles, recherche de sous-tableaux dont la somme vaut une cible, ou notion d'*equilibre* (indice pivot). Tres puissant combine a un hash map : on stocke les sommes prefixees deja vues pour compter les sous-tableaux en un seul passage.

Quand l'utiliser

Nombreuses requetes de somme sur intervalle ; compter les sous-tableaux de somme donnee (combine avec un hash map) ; indice pivot.

Complexite : Temps $O(n)$ precalcul, $O(1)$ requete | Espace $O(n)$

Problemes types : Range Sum Query (LC 303), Subarray Sum = K (LC 560), Pivot Index (LC 724).

```
def subarray_sum_k(nums, k):
    count = running = 0
    seen = {0: 1}                                # somme prefixee -> nb d'occurrences
    for x in nums:
        running += x
        count += seen.get(running - k, 0)
        seen[running] = seen.get(running, 0) + 1
    return count
```

Piege : initialise `seen = {0: 1}` pour compter les sous-tableaux qui commencent a l'indice 0.

7. Sorting (tri)

Trier revele souvent la structure cachee d'un probleme (voisinage, chevauchement). Cout de reference : $O(n \log n)$.

L'idée. Une fois les donnees ordonnees, les elements qui « vont ensemble » deviennent voisins : les doublons se touchent, les intervalles qui se chevauchent se suivent, une recherche dichotomique devient possible. Trier n'est pas la reponse en soi, mais l'*etape preparatoire* qui rend le vrai algorithme simple. En Python, `sort()` (Timsort) est stable et l'argument `key` permet de trier selon n'importe quel critere.

Reconnaitre le pattern. L'ordre des elements n'a pas d'importance dans l'enonce, mais un traitement devient evident *s'ils sont ordonnes* : fusionner des intervalles, detecter des doublons, planifier des reunions, preparer un two-pointers ou une dichotomie. Cout a assumer : $O(n \log n)$.

Quand l'utiliser

Regrouper ou detecter des doublons ; fusionner des intervalles ; preparer un two-pointers ; ordonner selon un critere personnalise.

Complexite : Temps $O(n \log n)$ | Espace $O(n)$ ou $O(1)$

Problemes types : Merge Intervals (LC 56), Meeting Rooms (LC 252), Largest Number (LC 179).

```
def merge_intervals(intervals):
    intervals.sort(key=lambda p: p[0])
    res = []
    for start, end in intervals:
        if res and start <= res[-1][1]:
            res[-1][1] = max(res[-1][1], end)  # fusionner
        else:
            res.append([start, end])
    return res
```

Piege : le tri detruit l'ordre initial. Si tu dois rendre des indices d'origine, trie des paires (valeur, indice).

8. Binary Search (recherche dichotomique)

Recherche en $O(\log n)$ sur un espace trie ou monotone. Cle : trouver un *predicat monotone* (vrai puis faux, ou l'inverse).

L'idée. A chaque etape, on regarde l'element du milieu et on elimine *la moitie* des candidats. Diviser l'espace de recherche par deux a chaque tour donne un cout logarithmique : chercher dans un million d'elements ne demande qu'une vingtaine de comparaisons. La version avancee ne cherche pas une valeur mais la *frontiere* d'une propriete monotone : « quelle est la plus petite valeur qui fonctionne ? ».

Reconnaitre le pattern. Donnees *triees*, ou reponse qui varie de facon *monotone* (si `x` marche, tout `x` plus grand marche aussi). Formulation type : « trouver la premiere / derniere position qui... », ou « minimiser une capacite / vitesse telle que... ». Des qu'on peut poser une question a reponse oui/non monotone, la dichotomie s'applique.

Quand l'utiliser

Tableau trie ; reponse monotone (« la valeur `x` fonctionne-t-elle ? ») ; recherche de la premiere ou derniere position valide.

Complexite : Temps $O(\log n)$ | Espace $O(1)$

Problem types : Binary Search (LC 704), Search in Rotated Array (LC 33), Koko Bananas (LC 875).

```
def binary_search(nums, target):
    left, right = 0, len(nums) - 1
    while left <= right:
        mid = left + (right - left) // 2    # evite le debordement
        if nums[mid] == target:
            return mid
        if nums[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

Piege : attention aux bornes ($<$ vs $<=$) et a la mise a jour de $\text{mid} \pm 1$ pour eviter les boucles infinies.

9. Stack (pile LIFO)

Last In, First Out. En Python : une simple `list` avec `append()` et `pop()`.

L’idee. La pile retient le dernier element ajoute pour le traiter en premier – exactement le comportement dont on a besoin quand des elements doivent etre « fermes » dans l’ordre inverse de leur ouverture (parentheses, balises), ou quand on veut suspendre une tache pour y revenir plus tard (DFS iteratif, annuler/refaire). Les operations **push** et **pop** sont en $O(1)$.

Reconnaitre le pattern. Correspondances imbriquees (parentheses, accolades, balises), evaluation d’expressions, parcours en profondeur sans recursion, ou tout probleme ou « le plus recent d’abord » est la bonne regle. Signal fort : on doit se souvenir d’elements *en attente* et les traiter dans l’ordre inverse.

Quand l’utiliser

Parentheses / expressions equilibrees ; parsing et evaluation ; DFS iteratif ; fonctionnalites annuler / refaire.

Complexite : Temps $O(1)$ par operation | Espace $O(n)$

Problem types : Valid Parentheses (LC 20), Min Stack (LC 155), Evaluate RPN (LC 150).

```
def valid_parentheses(s):
    pairs = {"(": ")", "[": "]", "{": "}"
    stack = []
    for c in s:
        if c in pairs:
            if not stack or stack.pop() != pairs[c]:
                return False
        else:
            stack.append(c)
    return not stack
```

Piege : pense a verifier que la pile est *vide* a la fin ; une pile non vide signale des ouvrants non fermes.

10. Monotonic Stack (pile monotone)

Une pile dont les elements restent toujours croissants ou decroissants. Chaque element est empile et depile au plus une fois : cout total $O(n)$.

L’idée. Quand on cherche, pour chaque element, « le prochain plus grand » (ou plus petit), une double boucle donne $O(n^2)$. La pile monotone garde seulement les candidats encore *utiles* : avant d’empiler un nouvel element, on depile tous ceux qu’il « bat », et c’est justement lui leur reponse. Comme chaque element entre et sort une seule fois, le cout total reste $O(n)$ malgre la boucle interne.

Reconnaitre le pattern. Formulations « prochain / precedent element plus grand / plus petit », spans (combien de jours jusqu’au prochain plus chaud), histogrammes, eaux de pluie. Astuce : on empile souvent des *indices* plutot que des valeurs, pour pouvoir calculer des distances.

Quand l’utiliser

Prochain element plus grand / plus petit ; largest rectangle in histogram ; trapping rain water ; spans boursiers.

Complexite : Temps $O(n)$ | Espace $O(n)$

Problemes types : Daily Temperatures (LC 739), Next Greater Element (LC 496), Largest Rectangle (LC 84).

```
def daily_temperatures(temps):
    res = [0] * len(temps)
    stack = []  # indices, temperatures decroissantes
    for i, t in enumerate(temps):
        while stack and temps[stack[-1]] < t:
            j = stack.pop()
            res[j] = i - j
        stack.append(i)
    return res
```

Piege : on empile souvent des *indices* plutot que des valeurs, pour pouvoir calculer des distances.

11. Backtracking (retour sur trace)

Explorer toutes les possibilites via un schema *choisir / recuser / annuler*. Cout souvent exponentiel.

L’idée. On construit une solution pas a pas. A chaque etape on fait un *choix*, on explore recursivement ce que ce choix entraine, puis on *annule* le choix (backtrack) pour essayer le suivant. On explore ainsi un *arbre de decisions*, et l’annulation permet de reutiliser la meme structure pour toutes les branches. On peut couper des branches (pruning) des qu’une solution partielle viole une contrainte, ce qui accelere beaucoup en pratique.

Reconnaitre le pattern. L’enonce demande *toutes* les solutions : combinaisons, permutations, sous-ensembles, ou un placement respectant des contraintes (Sudoku, N-Queens). Signal : « generer / enumerer tous les... ». Attention au cout, naturellement exponentiel ($O(2^n)$ ou $O(n!)$).

Quand l’utiliser

Toutes les combinaisons, permutations ou sous-ensembles ; problemes a contraintes (Sudoku, N-Queens) ; generation exhaustive.

Complexite : Temps $O(2^n)$ ou $O(n!)$ | Espace $O(n)$ pile de recursion

Problemes types : Subsets (LC 78), Permutations (LC 46), Combination Sum (LC 39).

```
def subsets(nums):
    res, path = [], []
    def backtrack(start):
        res.append(path[:])  # copie de l'etat courant
        for i in range(start, len(nums)):
            path.append(nums[i])  # choisir
```

```

        backtrack(i + 1)
        path.pop()          # annuler (backtrack)
    backtrack(0)
    return res

```

Piege : enregistre une *copie* (`path[:]`), sinon toutes les solutions pointeront vers la meme liste mutable.

12. Linked List (liste chaine)

Des noeuds relies par `.val` et `.next`. Les entretiens insistent sur la manipulation propre des pointeurs.

L’idee. Contrairement a un tableau, on ne peut pas sauter directement au *i*-eme element : il faut suivre les liens de proche en proche. Tout le savoir-faire consiste a manipuler quelques pointeurs (`prev`, `curr`, `next`) dans le bon ordre, sans perdre le reste de la liste. Deux techniques reviennent : le *noeud sentinelle* (dummy) qui simplifie la gestion de la tete, et les pointeurs *lent/rapide* pour le milieu ou la detection de cycle.

Reconnaitre le pattern. L’enonce fournit une liste chaine et demande de l’*inverser*, la *fusionner*, detecter un *cycle*, ou supprimer un noeud. Le vrai defi n’est pas l’algorithme mais la *rigueur* sur les cas limites : liste vide, un seul noeud, modification de la tete ou de la queue.

Quand l’utiliser

Inverser une liste ; detecter un cycle (lent/rapide) ; fusionner deux listes ; supprimer le *n*-ieme noeud depuis la fin.

Complexite : Temps $O(n)$ | Espace $O(1)$

Problemes types : Reverse List (LC 206), Linked List Cycle (LC 141), Merge Two Lists (LC 21).

```

def reverse_list(head):
    prev = None
    curr = head
    while curr:
        nxt = curr.next      # sauvegarder le suivant
        curr.next = prev    # inverser le lien
        prev = curr
        curr = nxt
    return prev

```

Piege : soigne les cas limites – liste vide, un seul noeud, mise a jour de `head`. Un *noeud sentinelle* (dummy) simplifie souvent le code.

13. Fast & Slow Pointers (lievre et tortue)

Deux pointeurs avancent a des vitesses differentes : **slow** d’un pas, **fast** de deux. Le rapide finit toujours par rattraper le lent *s’il existe un cycle*.

L’idee. En faisant courir un pointeur deux fois plus vite qu’un autre, on obtient deux informations gratuitement : s’ils se rejoignent, il y a un cycle ; et quand le rapide atteint la fin, le lent est pile au *milieu*. Aucune memoire supplementaire, un seul passage.

Reconnaitre le pattern. Detection de cycle (liste chaine, suite de nombres comme *happy number*), recherche du milieu d’une liste, ou detection d’un nombre duplique en voyant un tableau comme une liste de « pointeurs ».

Quand l'utiliser

Detecter/localiser un cycle ; trouver le milieu d'une liste ; happy number ; nombre duplique en $O(1)$ espace.

Complexite : Temps $O(n)$ | Espace $O(1)$

Problemes types : Linked List Cycle (LC 141/142), Middle of List (LC 876), Happy Number (LC 202), Find the Duplicate (LC 287).

```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next          # +1
        fast = fast.next.next     # +2
        if slow is fast:         # rattrapage -> cycle
            return True
    return False
```

Piege : teste `fast` and `fast.next` avant `fast.next.next`, sinon `NoneType` sur une liste de longueur paire.

14. Merge Intervals (fusion d'intervalles)

On trie les intervalles par debut, puis on fusionne ceux qui se chevauchent au fil d'un seul parcours.

L'idée. Une fois les intervalles ordonnes par leur debut, deux intervalles se chevauchent *ssi* le debut du courant est inferieur ou egal a la fin du dernier conserve. Il suffit alors d'etendre la fin, sinon d'ouvrir un nouvel intervalle. Le tri transforme un probleme quadratique en un balayage lineaire.

Reconnaitre le pattern. Tout enonce parlant de *plages*, *creneaux*, *reservations* ou *reunions* qui peuvent se recouvrir : fusionner, inserer, calculer une intersection, ou compter les ressources simultanees (salles).

Quand l'utiliser

Fusionner des plages qui se chevauchent ; inserer un intervalle ; intersection de deux listes ; nombre minimal de salles de reunion.

Complexite : Temps $O(n \log n)$ | Espace $O(n)$

Problemes types : Merge Intervals (LC 56), Insert Interval (LC 57), Interval Intersection (LC 986), Meeting Rooms II (LC 253).

```
def merge(intervals):
    intervals.sort(key=lambda p: p[0])          # trier par debut
    res = []
    for start, end in intervals:
        if res and start <= res[-1][1]:         # chevauchement
            res[-1][1] = max(res[-1][1], end)
        else:
            res.append([start, end])
    return res
```

Piege : pense au chevauchement « bord a bord » (`[1,4]` et `[4,5]`) : selon l'enonce, `<=` fusionne, `<` non.

15. Cyclic Sort (tri cyclique)

Quand un tableau contient des nombres d'une plage connue ($1..n$ ou $0..n$), on place chaque valeur a sa position attendue par echanges. Toute place mal remplie revele un manquant ou un duplique.

L'idée. La valeur v « appartient » a l'index $v-1$. En echangeant jusqu'a ce que chaque case soit bien remplie, on trie en $O(n)$ sans memoire. Ensuite, un simple balayage montre quels index ne correspondent pas : ce sont les valeurs absentes ou en double.

Reconnaitre le pattern. Tableau de n entiers dans $[1..n]$ ou $[0..n]$, et on cherche un *nombre manquant*, un *duplique*, ou *tous les disparus* – le tout idealement en espace constant.

Quand l'utiliser

Nombres dans une plage $[1..n]$; trouver le manquant, le duplique, ou tous les disparus en $O(1)$ espace.

Complexite : Temps $O(n)$ | Espace $O(1)$

Problemes types : Missing Number (LC 268), Find All Disappeared (LC 448), Set Mismatch (LC 645), First Missing Positive (LC 41).

```
def cyclic_sort(nums):
    i = 0
    while i < len(nums):
        correct = nums[i] - 1          # place attendue de nums[i]
        if nums[i] != nums[correct]:
            nums[i], nums[correct] = nums[correct], nums[i]
        else:
            i += 1
    return nums
```

Piege : n'incrémente i que lorsque l'element est deja a sa place, sinon tu sautes des valeurs mal placees.

16. BFS (parcours en largeur)

On explore le graphe *niveau par niveau* depuis une source, a l'aide d'une file (queue). Dans un graphe non pondere, BFS donne le *plus court chemin* en nombre d'aretes.

L'idée. Une file FIFO garantit qu'on visite les noeuds par distance croissante : d'abord les voisins directs, puis leurs voisins, etc. En traitant un niveau entier a la fois (boucle sur $\text{len}(\text{queue})$), on sait exactement a quelle profondeur on se trouve.

Reconnaitre le pattern. « Plus court chemin » / « nombre minimal d'etapes » dans une grille ou un graphe non pondere, parcours d'arbre par niveaux, ou propagation simultanee depuis plusieurs sources (multi-source BFS).

Quand l'utiliser

Plus court chemin non pondere ; parcours par niveaux ; distance minimale dans une grille ; propagation multi-source.

Complexite : Temps $O(V + E)$ | Espace $O(V)$

Problemes types : Binary Tree Level Order (LC 102), Rotting Oranges (LC 994), Shortest Path in Grid (LC 1091).

```
from collections import deque
def level_order(root):
```

```

if not root: return []
res, q = [], deque([root])
while q:
    level = []
    for _ in range(len(q)):          # tout le niveau courant
        node = q.popleft()
        level.append(node.val)
        if node.left: q.append(node.left)
        if node.right: q.append(node.right)
    res.append(level)
return res

```

Piege : marque un noeud comme visite *au moment de l'enfiler*, pas au defilement, sinon il peut entrer plusieurs fois dans la file.

17. DFS (parcours en profondeur)

On explore aussi loin que possible le long d'une branche avant de revenir en arriere (backtrack).
Implementation naturelle par recursion (pile d'appels).

L'idée. A partir d'un noeud, on plonge recursivement dans un voisin non visite, puis le suivant, etc. La pile d'appels memorise le chemin de retour. DFS epouse parfaitement les structures arborescentes et le decoupage en composantes connexes.

Reconnaitre le pattern. Compter/explorer des *regions* (iles dans une grille), enumerer des *chemins*, parcourir un arbre (pre/in/post-order), ou detecter des composantes connexes.

Quand l'utiliser

Composantes connexes ; nombre/aire d'iles ; chemins dans un arbre ou graphe ; exploration exhaustive d'une structure.

Complexite : Temps $O(V + E)$ | Espace $O(V)$ pile

Problemes types : Number of Islands (LC 200), Max Area of Island (LC 695), Preorder Traversal (LC 144).

```

def num_islands(grid):
    n, m = len(grid), len(grid[0])
    def sink(r, c):
        if 0 <= r < n and 0 <= c < m and grid[r][c] == "1":
            grid[r][c] = "0"          # marquer visite
            sink(r+1, c); sink(r-1, c); sink(r, c+1); sink(r, c-1)
    count = 0
    for r in range(n):
        for c in range(m):
            if grid[r][c] == "1":
                count += 1; sink(r, c)
    return count

```

Piege : sur de grands graphes, la recursion peut depasser la limite de pile Python – passe a une pile explicite si besoin.

18. Topological Sort (tri topologique)

Ordonne les sommets d'un graphe oriente *acyclique* (DAG) : pour chaque arete $u \rightarrow v$, u apparait avant v . Sert a ordonnancer des taches avec dependances.

L'idée (Kahn). On calcule le *degre entrant* de chaque sommet, puis on retire d'abord ceux qui n'ont aucune dependance (in-degree 0). Retirer un sommet decremente le degre de ses successeurs, qui deviennent a leur tour disponibles. Si l'on ne sort pas tous les sommets, c'est qu'il y a un *cycle*.

Reconnaitre le pattern. Dependances entre elements : ordre de cours (prerequis), ordre de compilation, planification de taches, resolution de contraintes « X doit venir avant Y ».

Quand l'utiliser

Ordonnancer des taches avec prerequis; detecter un cycle dans un graphe oriente; build/compilation; sequence de cours valide.

Complexite : Temps $O(V + E)$ | Espace $O(V)$

Problemes types : Course Schedule (LC 207), Course Schedule II (LC 210), Alien Dictionary (LC 269).

```
from collections import deque
def topo_order(n, edges):
    adj = {i: [] for i in range(n)}
    indeg = [0]*n
    for u, v in edges:
        # u -> v
        adj[u].append(v); indeg[v] += 1
    q = deque(i for i in range(n) if indeg[i] == 0)
    order = []
    while q:
        node = q.popleft(); order.append(node)
        for nxt in adj[node]:
            indeg[nxt] -= 1
            if indeg[nxt] == 0: q.append(nxt)
    return order if len(order) == n else [] # [] => cycle
```

Piege : un resultat plus court que n signale un cycle – toujours verifier cette condition avant d'utiliser l'ordre.

19. Union-Find / DSU (ensembles disjoints)

Structure qui regroupe des elements en ensembles disjoints avec deux operations quasi constantes : **find** (representant) et **union** (fusion).

L'idée. Chaque ensemble est un arbre pointant vers une racine. **find** remonte a la racine; **union** accroche une racine sous l'autre. Deux optimisations rendent le tout quasi $O(1)$: la *compression de chemin* (aplatir l'arbre pendant **find**) et l'*union par rang* (accrocher le petit arbre sous le grand).

Reconnaitre le pattern. Connexite *dynamique* (aretes ajoutees au fil de l'eau), comptage de composantes, detection de cycle dans un graphe non oriente, ou algorithme de Kruskal (arbre couvrant minimal).

Quand l'utiliser

Composantes connexes dynamiques; detecter un cycle (graphe non oriente); regrouper des elements; Kruskal (MST).

Complexite : Temps $O(\alpha(n)) \approx O(1)$ | Espace $O(n)$

Problemes types : Number of Provinces (LC 547), Redundant Connection (LC 684), Accounts Merge (LC 721).

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n)); self.rank = [0]*n
```

```
def find(self, x):
    while self.parent[x] != x:
        self.parent[x] = self.parent[self.parent[x]] # compression
        x = self.parent[x]
    return x
def union(self, a, b):
    ra, rb = self.find(a), self.find(b)
    if ra == rb: return False # deja connectes
    if self.rank[ra] < self.rank[rb]: ra, rb = rb, ra
    self.parent[rb] = ra
    if self.rank[ra] == self.rank[rb]: self.rank[ra] += 1
    return True
```

Piege : sans compression de chemin *et* union par rang, `find` peut degenerer en $O(n)$ sur une chaine.

20. Dynamic Programming (programmation dynamique)

On decompose un probleme en sous-problemes qui *se chevauchent*, et on memorise leurs solutions pour ne les calculer qu'une seule fois.

L'idée. Quand une solution recursive recalcule sans cesse les memes sous-problemes, on stocke chaque resultat (memoisation top-down) ou on remplit un tableau `dp` des petits cas vers les grands (bottom-up). Deux ingredients indispensables : *sous-structure optimale* (l'optimum se compose d'optima de sous-problemes) et *sous-problemes repetes*.

Reconnaitre le pattern. Formulations « nombre de facons de... », « cout min/max pour... », « est-ce possible... », choix sequentiels avec dependance sur les etapes precedentes (escaliers, pieces, sac a dos, sous-sequences).

Quand l'utiliser

« Nombre de facons » ; min/max cout ; possible/impossible ; choix avec sous-structure optimale et sous-problemes qui se repetent.

Complexite : Temps $O(\text{etats} \times \text{transition})$ | Espace $O(\text{etats})$

Problemes types : Climbing Stairs (LC 70), Coin Change (LC 322), House Robber (LC 198), LIS (LC 300), Knapsack.

```
def coin_change(coins, amount):
    dp = [float("inf")] * (amount + 1)
    dp[0] = 0
    for coin in coins:
        for x in range(coin, amount + 1):
            dp[x] = min(dp[x], dp[x - coin] + 1)
    return dp[amount] if dp[amount] != float("inf") else -1
```

Piege : definis clairement l'*etat* (que represente `dp[i]` ?) et le *cas de base* avant d'ecrire la transition – 90% des bugs viennent de la.

21. Greedy (algorithme glouton)

A chaque etape, on fait le choix *localement optimal* en esperant atteindre l'optimum global. Rapide, mais valable seulement si le probleme a la « propriete du choix glouton ».

L'idée. Contrairement a la DP, le glouton ne revient jamais en arriere : il s'engage sur le meilleur choix immediat. Ca ne fonctionne que si l'on peut *prouver* qu'un choix local optimal ne ferme pas la porte a l'optimum global (souvent apres un tri bien choisi).

Reconnaitre le pattern. Ordonnancement d'activités/intervalles, optimisation avec un critere simple a trier (fin la plus proche, plus gros d'abord), sauts, monnaie sur systemes canoniques.

Quand l'utiliser

Choisir/planifier des intervalles; jump game; probleme optimisable par un tri + choix local prouve sur; alternative rapide a la DP.

Complexite : Temps $O(n \log n)$ (souvent) | Espace $O(1)$

Problem types : Jump Game (LC 55/45), Non-overlapping Intervals (LC 435), Candy (LC 135), Best Time to Buy/Sell II (LC 122).

```
def can_jump(nums):
    reachable = 0
    for i, jump in enumerate(nums):
        if i > reachable:
            # trou infranchissable
            return False
        reachable = max(reachable, i + jump)
    return True
```

Piege : un glouton « evident » peut etre faux (ex. Coin Change avec des pieces non canoniques). En cas de doute, verifie avec la DP.

22. Bit Manipulation (manipulation de bits)

On opere directement sur la representation binaire des entiers. Souvent la solution la plus rapide et en espace constant.

L'idée. Les operateurs bit a bit (&, |, ^, ~, <<, >>) manipulent tous les bits en parallele. Quelques identites reviennent sans cesse : $x \oplus x = 0$ (le XOR annule les doublons), $x \& (x-1)$ efface le bit 1 le plus a droite, $x \& 1$ teste la parite.

Reconnaitre le pattern. Elements en double sauf un (XOR), comptage de bits, tests de puissance de deux, ensembles representes en bitmask, ou toute optimisation memoire/vitesse a base d'entiers.

Quand l'utiliser

Trouver l'unique non double (XOR); compter les bits; puissance de deux; masques de bits (bitmask) pour representes des ensembles.

Complexite : Temps $O(n)$ ou $O(1)$ | Espace $O(1)$

Problem types : Single Number (LC 136), Number of 1 Bits (LC 191), Power of Two (LC 231), Counting Bits (LC 338).

```
def single_number(nums):
    result = 0
    for x in nums:
        result ^= x
    return result
    # les doublons s'annulent, reste l'unique

def count_ones(n):
    count = 0
    while n:
        n &= n - 1
        count += 1
    return count
    # efface le 1 le plus a droite
```


Piege : en Python les entiers ont une taille illimitée ; pour simuler du 32 bits, masque avec `& 0xFFFFFFFF`.

23. Fiche recapitulative des complexites

A memoriser avant l'entretien :

Pattern	Temps	Espace
Hash Map / Set	$O(1)$ moyen	$O(n)$
Two Pointers	$O(n)$	$O(1)$
Sliding Window	$O(n)$	$O(k)$
Prefix Sum	$O(n)$ / $O(1)$	$O(n)$
Sorting	$O(n \log n)$	$O(n)$
Binary Search	$O(\log n)$	$O(1)$
Stack	$O(n)$	$O(n)$
Monotonic Stack	$O(n)$	$O(n)$
Backtracking	$O(2^n)$ / $O(n!)$	$O(n)$
Linked List	$O(n)$	$O(1)$
Fast & Slow Pointers	$O(n)$	$O(1)$
Merge Intervals	$O(n \log n)$	$O(n)$
Cyclic Sort	$O(n)$	$O(1)$
BFS	$O(V + E)$	$O(V)$
DFS	$O(V + E)$	$O(V)$
Topological Sort	$O(V + E)$	$O(V)$
Union-Find (DSU)	$O(\alpha(n))$	$O(n)$
Dynamic Programming	$O(\text{etats} \times \text{tr.})$	$O(\text{etats})$
Greedy	$O(n \log n)$	$O(1)$
Bit Manipulation	$O(n)$ / $O(1)$	$O(1)$

Checklist du jour J

- Parler a voix haute – le raisonnement est note.
- Clarifier les contraintes AVANT de coder.
- Annoncer la complexite temps + espace.
- Traiter les cas limites (vide, 1 element, negatifs, debordement).
- Nommer clairement ; code lisible > code astucieux.
- Bloque ? Revenir a la force brute, puis optimiser.
- Tester a la main avant de dire « c'est bon ».